Multi-Agent DevOps Incident Analysis Suite

Automated Incident Response Powered by AI Agents

Group 08 — AI Engineers Accelerated Program C5

LangGraph • GPT-4o • Streamlit • Slack SDK • pytest

April 2026

The Problem

DevOps teams drown in heterogeneous logs during incidents. Manual triage is slow, error-prone, and wastes critical response time.

Slow Triage

Engineers spend 30-60 min manually scanning logs across services

Format Chaos

Syslog, JSON, CSV, plain text — no unified parsing pipeline

Alert Fatigue

Low-priority noise buries genuinely critical incidents

No Runbooks

Remediation knowledge lives in engineers' heads, not systems

Solution Concept

An AI-powered multi-agent system that ingests raw ops logs, classifies incidents by severity, generates root-cause analysis and remediations, creates runbooks, and pushes targeted alerts — all in seconds.

5 Specialized AI Agents

Classifier → Remediation → Cookbook → Slack → JIRA — each expert in its domain

Conditional Orchestration

LangGraph routes agents dynamically based on severity — CRITICAL triggers all 5, LOW only gets a runbook

Multi-Format Ingestion

Syslog, JSON, CSV, plain text — handles any log format your stack produces

Real-Time Execution Trace

Full pipeline timeline showing which agents ran, per-agent timing, and I/O summaries

Architecture

Severity-Based Conditional Fan-Out (LangGraph add_conditional_edges)

Technical Highlights

1. LangGraph StateGraph with conditional fan-out edges and parallel execution
2. Shared IncidentState TypedDict with operator.add for trace aggregation
3. Centralized config.py for severity constants — no duplication across agents
4. Structured logging (Python logging module) in graph nodes and agents
5. LLM JSON parsing with markdown-stripping and empty-input guard clauses
6. Real Slack SDK (WebClient + Block Kit) — not a mocked HTTP call

Code Quality & Organization

Project Structure

```
├── app.py            # Entry point
├── config.py         # Centralized constants
├── orchestrator/
│   ├── graph.py      # StateGraph + logging
│   ├── state.py      # TypedDict models
│   └── router.py     # Severity routing
├── agents/           # 5 AI agents
│   ├── classifier.py # + docstrings, guards
│   ├── remediation.py
│   ├── cookbook.py
│   ├── slack_notifier.py
│   └── jira_ticket.py
├── ui/               # Streamlit frontend
├── utils/            # Helpers
├── tests/            # 9 test files
└── sample_logs/      # 4 demo datasets
```

Quality Practices

- ✓ **Separation of Concerns**
  Orchestrator / Agents / UI / Utils / Config
- ✓ **Typed State Management**
  TypedDict + Annotated operators
- ✓ **Single Responsibility**
  Each agent = 1 file, 1 function, 1 purpose
- ✓ **Centralized Config**
  config.py eliminates constant duplication
- ✓ **Structured Logging**
  Python logging module in all graph nodes
- ✓ **Comprehensive Testing**
  9 test files with mocked LLM responses
- ✓ **Defensive Guards**
  Empty-input checks before every LLM call
- ✓ **Secrets via .env**
  Zero hardcoded keys or connection strings

Documentation

README.md

Architecture, quick start, usage guide, agent specs, routing table, sample logs

Architecture Design Spec

Agent contracts, state schema, routing logic, dashboard wireframes

Implementation Plan

Task breakdown with milestones, testing strategy, deployment checklist

Docstrings & Type Hints

Google-style docstrings on all agent functions with Args/Returns

.env.example + config.py

Self-documenting env template + centralized constants file

Test Suite as Documentation

9 test files demonstrate expected behavior and edge cases

UI / UX Design

Dark Theme

GitHub Dark + New Relic. Built for engineers working incidents at 2 AM.

Severity Colors

Red → Orange → Blue → Gray. Instant visual triage at a glance.

6-Tab Dashboard

Analysis · Remediations · Cookbook · Slack · JIRA · Trace

Trace Timeline

Visual agent execution bar with per-node timing and status.

Glassmorphic cards • Severity badges • Confidence bars • Category tags • Expandable trace details • Responsive sidebar

Live demo follows →

Live Demo

1. **Upload Logs** — ~10s
   Upload mixed_incident.log — 15 entries: OOM, timeouts, auth failures, disk warnings
2. **Watch Pipeline Run** — ~15s
   Spinner shows multi-agent pipeline processing: Classifier → Remediation → Router → Fan-out
3. **Review Analysis** — ~20s
   Color-coded severity cards — CRITICAL red, HIGH orange, MEDIUM blue, LOW gray
4. **Explore Remediations** — ~20s
   Root cause analysis with confidence scores, ordered fix steps, linked log entries
5. **Check Runbook** — ~15s
   Auto-generated markdown checklist grouped by priority — ready for on-call use
6. **Show Agent Trace** — ~10s
   Execution timeline with all 5 agents, per-agent timing, input/output summaries

Technology Stack

AI / LLM

- LangGraph 0.4.1
- LangChain 0.3.25
- GPT-4o (OpenRouter)
- Few-shot JSON prompts

Frontend

- Streamlit 1.45.1
- Custom CSS theme
- HTML components
- 6-tab dashboard

Integrations

- Slack SDK 3.34.0
- Block Kit messages
- JIRA (mocked)
- Multi-format parser

Quality & DevOps

- pytest 8.3.5
- unittest.mock
- Structured logging
- config.py constants

~20 source files • 9 test files • 5 AI agents • 7 dependencies • 4 log formats

Thank You

Group 08 — AI Engineers Accelerated Program C5

5	9	4	<30s
AI Agents	Test Files	Log Formats	Analysis Time

Questions?